Topic 26 - Database Design for Automation (SQLite, PostgreSQL & MySQL)

Step 1: Initialize Database Schema (Zero-Blocker In-Memory / SQLite Setup)

1. Create a new workflow named `Topic 11 - Database Design for Automation`. 2. Add a **Manual Trigger Node** named `Manual_Trigger`.
3. Connect a **Code Node** named `Code_InitRelationalDB`:

   Language: `JavaScript`

   This node creates and seeds our relational database tables:

```
// Relational Database Engine Simulation inside n8n static data
const staticData = $getWorkflowStaticData('global');

// 1. DDL: Create Tables & Constraints
staticData.tables = staticData.tables || {};

staticData.tables.customers = [
 { customerId: "CUST-001", name: "Acme Corp", creditLimit: 5000.00, currentBalance:
1200.00 },  { customerId: "CUST-002", name: "Globex Inc", creditLimit: 500.00,
currentBalance: 450.00 } ];

staticData.tables.orders = [];
staticData.tables.order_items = [];

return {
 json: {
 status: "DATABASE_INITIALIZED",
 schema: {
 tables: ["customers", "orders", "order_items"],
 constraints: ["FK_order_items_orders", "CHECK_credit_limit"],
 indexes: ["idx_orders_customer", "idx_items_order"]
 },
 timestamp: new Date().toISOString()
 }
};
```

Step 2: Stage Incoming Order Payload (`Code_StageIncomingOrder`)

1. **Create Workflow & Manual Trigger:**

   In your n8n dashboard, click **+ Add workflow** in the top-right corner.

   Rename to: `Topic 11 - Database Design for Automation (SQLite, PostgreSQL & MySQL)`.

   Click **+ Add first step...**, search for **`Manual Trigger`**, and place it.
   Rename to: `Manual_Trigger`.

2. **Add Order Payload Stager Node:**

   From `Manual_Trigger`, click `+`, search for `Code`, and add the node.
   Rename to: `Code_StageIncomingOrder`.

3. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

   Mode: Select `Run Once for All Items`.

   JavaScript Code: Paste the incoming multi-item order:

```
return [
 {
 json: {
 orderId: "ORD-DB-5001",
 customerId: "CUST-101",
 orderDate: new Date().toISOString(),
 lineItems: [
 { sku: "SKU-SRV-01", description: "Cloud Managed Server", quantity: 2, unitPrice: 450.00
},  { sku: "SKU-LIC-02", description: "Enterprise Security Seat", quantity: 5, unitPrice:
75.00 }  ]
 }
 }
];
```

4. **Test Node Checkpoint:**

   Click **Test step**. Verify staged order object.

Step 3: Insert Order Header with Parameterized Validation (Native Postgres or SQLite Sandbox)

Approach A: Native PostgreSQL Node (`Postgres_InsertOrderHeader`)

1. From `Code_StageIncomingOrder`, click `+`, search for `Postgres`, and select the **PostgreSQL** node (`n8n-nodes-base.postgres`).
2. Rename to: `Postgres_InsertOrderHeader`.
3. **Parameters Tab:**

   Credential to connect with: Select your PostgreSQL database credential.

   Operation: Select `Execute Query`.

   Query: Enter parameterized SQL with positional placeholders:

```
INSERT INTO orders (order_id, customer_id, order_date,
total_amount, status) VALUES ($1, $2, $3, $4, $5)
RETURNING order_id, status;
```

   Query Parameters: In the parameters list, map values:

   `$1`: `={{ $json.orderId }}` (Mode: **Expression (`fx`)**)

   `$2`: `={{ $json.customerId }}` (Mode: **Expression (`fx`)**)

   `$3`: `={{ $json.orderDate }}` (Mode: **Expression (`fx`)**)

   `$4`: `={{ $json.lineItems.reduce((acc, it) => acc +`

`(it.quantity * it.unitPrice), 0) }}`

`$5`: `'PENDING'` (Fixed String)

Approach B: Zero-Blocker In-Memory / SQLite Sandbox (`Code_InsertOrderHeader`)

1. From `Code_StageIncomingOrder`, add a **Code** node named `Code_InsertOrderHeader`. 2. **Parameters Tab:**

Language: Select `JavaScript`.

Mode: Select `Run Once for Each Item`.

JavaScript Code: Paste:

```
const staticData = $getWorkflowStaticData('global');
staticData.orders = staticData.orders || [];

const order = $input.item.json;
const totalAmount = (order.lineItems || []).reduce((acc, it) => acc + (it.quantity * it.unitPrice), 0);

const orderRecord = {
 orderId: order.orderId,
 customerId: order.customerId,
 orderDate: order.orderDate,
 totalAmount: totalAmount,
 status: "PENDING"
};

staticData.orders.push(orderRecord);

return {
 json: {
 ...order,
 headerInserted: true,
 totalAmount: totalAmount
 }
};
```

3. **Test Node Checkpoint:**

Click **Test step**. Verify `headerInserted: true` and calculated `totalAmount: 1275`.

Step 4: Batch Insert Child Order Items (`Code_InsertOrderItems`)

1. **Add Node to Canvas:**

From `Code_InsertOrderHeader` (or `Postgres_InsertOrderHeader`), click `+`, search for `Code`, and add the node.

Rename to: `Code_InsertOrderItems`.

2. **Configure Parameters (Parameters Tab):**

Language: Select `JavaScript`.

Mode: Select `Run Once for Each Item`.

JavaScript Code: Paste:

```
const staticData = $getWorkflowStaticData('global');
staticData.orderItems = staticData.orderItems || [];

const order = $input.item.json;
const lineItems = order.lineItems || [];

const insertedItems = lineItems.map((it, idx) => {
 const record = {
 itemId: `ITEM-${order.orderId}-${idx + 1}`,
 orderId: order.orderId,
 sku: it.sku,
 description: it.description,
 quantity: it.quantity,
 unitPrice: it.unitPrice,
 lineTotal: Number((it.quantity * it.unitPrice).toFixed(2))
 };
 staticData.orderItems.push(record);
 return record;
});

return {
 json: {
 orderId: order.orderId,
 customerId: order.customerId,
 totalAmount: order.totalAmount,
 itemsCount: insertedItems.length,
 items: insertedItems
 }
};
```

3. Test Node Checkpoint:

Click **Test step**. Verify both line items are inserted with foreign key references.

Step 5: Enforce Credit Limit & Commit Transaction (`Code_UpdateCustomerCredit`)

1. Add Node to Canvas:

From `Code_InsertOrderItems`, click `+`, search for `Code`, and add the node.

Rename to: `Code_UpdateCustomerCredit`.

2. Configure Parameters (Parameters Tab):

Language: Select `JavaScript`.

Mode: Select `Run Once for Each Item`.

JavaScript Code: Paste the ACID transaction commit/rollback simulator:

```
const staticData = $getWorkflowStaticData('global');
staticData.customers = staticData.customers || [
 { customerId: "CUST-101", name: "Apex Global", balance: 5000.00, creditLimit: 10000.00 }];
```

```
const order = $input.item.json;
```

```
const customer = staticData.customers.find(c => c.customerId === order.customerId);

if (!customer) {
 return { json: { success: false, reason: "CUSTOMER_NOT_FOUND" } };
}

const newBalance = customer.balance + order.totalAmount;

// Enforce constraint: balance cannot exceed credit limit
if (newBalance > customer.creditLimit) {
 // Rollback transaction: delete orphaned order header and child items
 staticData.orders = staticData.orders.filter(o => o.orderId !== order.orderId);
staticData.orderItems = staticData.orderItems.filter(i => i.orderId !==
order.orderId);

 return {
 json: {
 transactionStatus: "ROLLED_BACK",
 reason: "CREDIT_LIMIT_EXCEEDED",
 currentBalance: customer.balance,
 creditLimit: customer.creditLimit,
 attemptedCharge: order.totalAmount
 }
 };
}

// Commit transaction
customer.balance = newBalance;
const committedOrder = staticData.orders.find(o => o.orderId ===
order.orderId); if (committedOrder) committedOrder.status =
"COMMITTED";

return {
 json: {
 transactionStatus: "COMMITTED",
 orderId: order.orderId,
 customerId: customer.customerId,
 updatedBalance: newBalance,
 committedAt: new Date().toISOString()
 }
};
```

3. **Test Node Checkpoint:**

   Click **Test step**. Verify `transactionStatus: "COMMITTED"`.

Step 6: Route Transaction Status (`IF_TransactionSucceeded`)

1. **Add Node to Canvas:**

   From `Code_UpdateCustomerCredit`, click `+`, search for `If`, and select the **If** node. Rename to: `IF_TransactionSucceeded`.

2. **Configure Parameters (Parameters Tab):**

   Click **Add Condition**.

   Value 1: Switch toggle to **Expression (`fx`)** and enter:

```
={{ $json.transactionStatus }}
```

Operation: Select `String` $ ightarrow$ `Equal`.

Value 2: Enter `COMMITTED` (Fixed).

3. **Connect False Branch:**

   From **false (bottom) output handle**, add a **Code** node named `Log_RollbackAudit`. Return: `{ json: { status: "TRANSACTION_ABORTED_SAFELY", details: $json } }`. Click **Test step**.

Step 7: Execute Indexed SQL JOIN (`Code_QueryFullOrderJoin`)

1. **Add Node to Canvas (True Branch):**

   Hover over the **true (top) output handle** of `IF_TransactionSucceeded`, click `+`, search for `Code`, and add the node.

   Rename to: `Code_QueryFullOrderJoin`.

2. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

   Mode: Select `Run Once for Each Item`.

   JavaScript Code: Paste the relational JOIN simulator:

```
const staticData = $getWorkflowStaticData('global');
const orderId = $json.orderId;

const order = (staticData.orders || []).find(o => o.orderId === orderId);
const customer = (staticData.customers || []).find(c => c.customerId ===
order?.customerId); const items = (staticData.orderItems || []).filter(i =>
i.orderId === orderId);

return {
 json: {
 orderId: order.orderId,
 status: order.status,
 customer: {
 id: customer.customerId,
 name: customer.name,
 balance: customer.balance
 },
 orderDate: order.orderDate,
 totalAmount: order.totalAmount,
 lineItems: items
 }
};
```

3. **Test Node Checkpoint:**

   Click **Test step**. Verify joined output.

Step 8: Build Final Transaction Audit (`Code_FinalTransactionAudit`)

1. **Add Node to Canvas:**

   From `Code_QueryFullOrderJoin`, click `+`, search for `Code`, and add the node. Rename to: `Code_FinalTransactionAudit`.

2. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

   Mode: Select `Run Once for Each Item`.

   JavaScript Code: Paste:

```
const item = $input.item.json;
return {
 json: {
 auditStatus: "DATABASE_TRANSACTION_VERIFIED",
 orderId: item.orderId,
 persistedTotal: item.totalAmount,
 lineItemCount: item.lineItems.length,
 auditTimestamp: new Date().toISOString()
 }
};
```

3. **Test Node Checkpoint:**

   Click **Test step**.